

Delhi Metro Route Finder (DSA Project in C++)

This project uses Dijkstra's Algorithm to find the shortest path between metro stations. Each station has a unique ID and name, and connections represent distances between stations.

```
#include <bits/stdc++.h>
using namespace std;

unordered_map<int, vector<pair<int,int>>> adj;
unordered_map<int, string> stationName;
unordered_map<string, int> stationID;

void addStation(int id, string name) {
    stationName[id] = name;
    stationID[name] = id;
}

void addEdge(int u, int v, int wt) {
    adj[u].push_back({v, wt});
    adj[v].push_back({u, wt});
}

void printPath(int dest, vector<int> &parent) {
    if(dest == -1) return;
    printPath(parent[dest], parent);
    cout << stationName[dest] << " -> ";
}

void dijkstra(int src, int dest, int n) {
    vector<int> dist(n+1, INT_MAX), parent(n+1, -1);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

    dist[src] = 0;
    pq.push({0, src});

    while(!pq.empty()) {
        auto [d, node] = pq.top();
        pq.pop();

        for(auto &nbr : adj[node]) {
            int next = nbr.first;
            int wt = nbr.second;

            if(dist[node] + wt < dist[next]) {
                dist[next] = dist[node] + wt;
                parent[next] = node;
                pq.push({dist[next], next});
            }
        }
    }

    cout << "\nShortest Distance: " << dist[dest] << endl;
    cout << "Path: ";
    printPath(dest, parent);
    cout << "END\n";
}

int main() {
    addStation(1, "Rajiv Chowk");
    addStation(2, "Kashmere Gate");
    addStation(3, "Central Secretariat");
    addStation(4, "Karol Bagh");

    addEdge(1,2,5);
    addEdge(1,3,3);
    addEdge(2,4,7);

    int choice;
    while(true) {
        cout << "\n1. Find Shortest Route\n2. Exit\nEnter choice: ";
        cin >> choice;

        if(choice == 1) {
            string src, dest;
```

```

        cout << "Enter Source Station: ";
        cin.ignore();
        getline(cin, src);
        cout << "Enter Destination Station: ";
        getline(cin, dest);

        if(stationID.count(src) && stationID.count(dest)) {
            dijkstra(stationID[src], stationID[dest], 4);
        } else {
            cout << "Invalid station name!\n";
        }
    } else {
        break;
    }
}
}

```